

NumPy

NumPy

Introduzione alla libreria NumPy per il calcolo numerico in Python.

Perché è utile usare NumPy

Le liste Python vanno bene per la maggior parte dei casi. Ma quando hai bisogno di fare **calcoli matematici su migliaia o milioni di numeri**, le liste diventano lente. NumPy risolve questo problema: fa le stesse operazioni ma in modo molto più veloce.

NumPy è alla base di quasi tutto l'ecosistema scientifico di Python: pandas, matplotlib, scikit-learn usano tutti NumPy internamente.

Installazione e importazione

```
pip install numpy
```

Per convenzione, NumPy si importa sempre con il soprannome `np` :

Gli array di NumPy

La struttura centrale di NumPy è l'**array** (`ndarray`) — simile a una lista Python, ma pensata per i calcoli numerici.

```
# Crea un array da una lista Python
a = np.array([1, 2, 3, 4, 5])
print(a)          # [1 2 3 4 5]
print(type(a))    # <class 'numpy.ndarray'>
print(a.dtype)    # int64 – il tipo dei dati (tutti interi a 64 bit)
print(a.shape)    # (5,) – la forma: 5 elementi in una dimensione

# Crea una matrice (array a due dimensioni)
m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
print(m)
print(m.shape)    # (3, 3) – 3 righe, 3 colonne
```

Creare array già pronti

```
print(np.zeros(5))          # [0. 0. 0. 0. 0.] – cinque zeri
print(np.ones(4))           # [1. 1. 1. 1.] – quattro uni
print(np.zeros((2, 3)))     # matrice 2x3 di zeri
print(np.eye(3))            # matrice identità 3x3 (uno sulla diagonale)
print(np.arange(0, 10, 2))  # [0 2 4 6 8] – come range(), ma per array
print(np.linspace(0, 1, 5)) # [0. 0.25 0.5 0.75 1.] – 5 punti equidistanti
                             # tra 0 e 1
print(np.random.rand(3))    # 3 numeri decimali casuali tra 0 e 1
```

La potenza delle operazioni vettoriali

Con le liste Python, per moltiplicare ogni elemento per 2 dovresti scrivere un ciclo. Con NumPy, le operazioni si applicano **all'intero array in una volta sola**:

```
a = np.array([1, 2, 3, 4, 5])

# Operazioni su tutti gli elementi – senza scrivere cicli!
print(a + 10)    # [11 12 13 14 15]
print(a * 2)     # [ 2  4  6  8 10]
print(a ** 2)    # [ 1  4  9 16 25]
print(a > 3)     # [False False False  True  True]

# Operazioni tra due array (elemento per elemento)
b = np.array([10, 20, 30, 40, 50])
print(a + b)     # [11 22 33 44 55]
print(a * b)     # [ 10  40  90 160 250]
```

Questo approccio è detto **vettorializzazione**: invece di fare le operazioni una alla volta con un ciclo, NumPy le fa tutte insieme in modo ottimizzato.

Funzioni matematiche integrate

```
a = np.array([1, 4, 9, 16, 25])

print(np.sqrt(a))    # [1.  2.  3.  4.  5.] – radice quadrata di ogni elemento
print(np.sum(a))     # 55    – somma di tutti gli elementi
print(np.mean(a))    # 11.0  – media
print(np.min(a))     # 1     – valore minimo
print(np.max(a))     # 25    – valore massimo
print(np.std(a))     # deviazione standard
```

Accedere agli elementi

```
m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

print(m[0])          # [1 2 3]          ← prima riga intera
print(m[1, 2])       # 6                ← riga 1, colonna 2
print(m[:, 0])       # [1 4 7]         ← tutta la colonna 0
print(m[0:2, 1:3])   # [[2 3] [5 6]]   ← un sotto-blocco (righe 0-1, colonne 1-2)
```

NumPy vs liste Python: la differenza di velocità

```
n = 1_000_000    # Un milione di numeri

# Metodo 1: lista Python con ciclo
lista = list(range(n))
inizio = time.time()
risultato = [x * 2 for x in lista]
print(f"Lista Python: {time.time() - inizio:.3f} secondi")

# Metodo 2: array NumPy
array = np.arange(n)
inizio = time.time()
risultato = array * 2
print(f"NumPy: {time.time() - inizio:.3f} secondi")
```

NumPy è tipicamente **10-100 volte più veloce** delle liste Python per operazioni numeriche. Su un milione di elementi, la differenza è notevole.

Quando usare NumPy vs liste

Situazione	Usa...
Dati misti (stringhe, numeri, oggetti)	Lista Python
Calcoli matematici su molti numeri	NumPy
Semplice raccolta di elementi	Lista Python
Matrici, algebra lineare, statistiche	NumPy
Qualsiasi libreria scientifica (pandas, matplotlib)	NumPy (lo usano internamente)